Java generics

Contents

What generics are and the problem they solve, how to write a generic class, the naming conventions, type erasure, and the PECS wildcard rule. Definitions stay tight; the answer does the teaching.

1. What are generics in Java, and what problem do they solve?

Generics let us write classes, interfaces, and methods that work across different data types while keeping **compile-time type safety**. They arrived in Java 5 to remove explicit casting and cut down on `ClassCastException`s.

Why they were introduced: before Java 5, collections stored everything as `Object`.

```
List list = new ArrayList();

list.add("Rohit");
list.add(100);
```

That was all allowed, because every class extends `Object`. The cost showed up on the way out. Reading a value needed an explicit cast:

```
String name = (String) list.get(0);
```

And if the cast did not match the actual type, it blew up at runtime:

```
String value = (String) list.get(1); // throws ClassCastException
```

The compiler could not catch the mistake. We only found out when the code ran.

With generics: we state the type once, and the compiler enforces it.

```java
List<String> names = new ArrayList<>();
names.add("Rohit");
names.add(100); // compilation error
```

Now the wrong type is rejected at compile time, and reading a value needs no cast:

```java
String name = names.get(0);
```

So generics move the error from runtime to compile time, and drop the casting noise along the way.

2. What are the advantages and disadvantages of generics?

Advantages:

Advantage	Explanation
Compile-time type safety	Type mismatches are caught during compilation, not at runtime
No explicit casting	Retrieved objects already have the right type
Fewer `ClassCastException`s	Invalid assignments are rejected by the compiler
Code reusability	One class or method works across many data types
Better readability	The type is explicit, so the code is easier to follow
Better API design	Enables reusable, type-safe APIs like `List<T>`, `Map<K,V>`, and `Optional<T>`

Disadvantages:

Disadvantage	Explanation
Type erasure	The generic type information is removed at runtime
No primitive types	We must use wrappers like `Integer` instead of `int`
No generic arrays	`new T[10]` is not allowed
Cannot instantiate a type parameter	`new T()` is illegal
No `instanceof` with parameterised types	`obj instanceof List<String>` is invalid
Can add complexity	Advanced generics (`? extends`, `? super`, nested generics) get hard to read

3. What is a generic class, and how do you create one?

A generic class is a class **parameterised with one or more type parameters**, so it can work with different data types while keeping compile-time type safety.

How we create one: put a type parameter inside angle brackets after the class name.

```
class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

Here `T` is the type parameter, and the actual type is supplied when we create the object, for example `Box<String>` or `Box<Integer>`.

Using it:

```
Box<String> box = new Box<>();
box.set("Java");

String value = box.get();
System.out.println(value);
```

Output:

```
Java
```

The `get()` returns a `String` directly, so no casting is required.

4. How many type parameters can a generic class have?

There is no language limit. A generic class can take one or more type parameters, though in practice one to three is the norm. If we find ourselves reaching for many, that is usually a hint that the design can be simplified.

One type parameter:

```
class Box<T> {
    private T value;
}
```

```
Box<String> box = new Box<>();
```

Two type parameters:

```
class Pair<K, V> {
    private K key;
    private V value;
}
```

```
Pair<String, Integer> pair = new Pair<>();
```

Three type parameters:

```
class Triple<A, B, C> {
    private A first;
    private B second;
    private C third;
}
```

```
Triple<String, Integer, Double> data = new Triple<>();
```

5. What naming conventions are used for generic type parameters (T, E, K, V, N, R)?

By convention, type parameters are **single uppercase letters**. They are not Java keywords, they are just names picked to make generic code easier to read.

Type parameter	Meaning	Common usage
T	Type	A general-purpose type
E	Element	Collections (`List<E>` , `Set<E>`)
K	Key	Maps (`Map<K, V>`)
V	Value	Maps (`Map<K, V>`)
N	Number	Numeric types (less common)
R	Result or return type	Generic methods, functional interfaces
U	Second type	When another generic type is needed
S	Second type	An additional generic type
X	Exception	A generic exception type (rare)

6. If generics provide type safety, why are they removed at runtime?

Generics give us type safety at **compile time**, but they are erased at runtime through **type erasure**, to keep backward compatibility with Java from before version 5.

Why: when Java 5 shipped generics in 2004, millions of applications were already running without them, full of raw code like this:

```
List list = new ArrayList();
```

If generics had changed the JVM or the class-file format, all that existing code and every older library would have broken. To avoid that, the designers chose type erasure: the compiler checks the types and then strips the generic information, so the compiled bytecode looks much like the pre-generics version. We get the safety at compile time without changing what the JVM has to run.

7. What is PECS?

PECS stands for **Producer Extends, Consumer Super**. It is the rule for choosing between `? extends T` and `? super T` when we use wildcards:

- **Producer, use** `extends`: if we only **read** values out of a structure, declare it `? extends T`.
- **Consumer, use** `super`: if we only **write** values into a structure, declare it `? super T`.

So if the collection produces items for us, use `extends`; if it consumes items we give it, use `super`. That one line keeps wildcard code both flexible and type-safe.